Binary Adaptive Paging:
Power-of-Two Region Decomposition with XOR-Based Address Lookup

Student: Abdulmajid alawad
Course: Operating Systems / Memory Management
Instructor: [Instructor Name]
Date: [Submission Date]

Abstract

Binary Adaptive Paging (BAP) decomposes virtual-memory regions into descending power-of-two logical chunks, with a compact XOR/MSB rule for identifying the chunk containing an address. Prior simulations (v2-v4) study structural decomposition and synthetic workloads; v5 audits native page-size and virtual-alignment eligibility; v6 verifies software-model physical-frame translation and permissions; and v7 accounts for hypothetical metadata bytes and binary-search comparisons. We extend the evaluation with v8, a C++20 software-lookup microbenchmark comparing BAP with globally searched capped and unbounded contiguous ranges and a stronger directly indexed capped-range baseline. Two independent pilot executions each produced 540 synthetic scenario/trace measurements. For a 10,000-page region with uniform-random accesses, BAP was slower than indexed ranges under contiguous backing (3.36 vs. 2.33 ns/lookup) but faster with deterministic 32-page physical fragmentation (27.66 vs. 31.53 ns/lookup). Under combined fragmentation and permission changes, the medians were nearly equal. The sampled machine was virtualized and inter-run variation was material, so these results are exploratory, not stable performance claims. A separate local execution passed correctness validation and generated 540 rows, but its CSV measurements were not supplied for aggregation. BAP has no demonstrated universal metadata or translation-performance advantage; hardware-TLB evaluation remains future work.

Keywords: virtual memory, Binary Adaptive Paging, XOR/MSB lookup, region indexing, contiguous ranges, indexed range baseline, software microbenchmark, memory metadata, preliminary performance evaluation

Contents

- 1. Introduction
- 2. Background and Motivation
- 3. Binary Adaptive Paging Design
- 4. Formal Model and Lookup Rule
- 5. Correctness Argument
- 6. Simulation Methodology
- 7. Results
- 8. Discussion
- 9. Limitations and Threats to Validity
- 10. Related Work
- 11. Conclusion
- References
- Appendix A. Reproducibility Notes

1. Introduction

Virtual memory is one of the central abstractions in modern operating systems. It allows each process to observe a private address space while the operating system and hardware cooperate to translate virtual addresses into physical locations. The most common practical design is page-based translation with a fixed base page size, such as 4 KB. Fixed-size paging is simple, well supported by hardware, and convenient for protection and demand paging. However, it also means that a large region may require a large number of page-table entries and many small translations in the translation lookaside buffer (TLB).

Large pages, huge pages, and superpages address this problem by allowing one translation entry to cover a larger memory range. This increases TLB coverage and can reduce page-table overhead. However, large pages are not a free solution. They often require suitable alignment and physical contiguity, can amplify page faults when access is sparse, and may interact badly with protection boundaries or copy-on-write. Transparent huge-page mechanisms attempt to make this process automatic, but they still need promotion, demotion, and split policies.

This paper explores a different point in the design space. Binary Adaptive Paging (BAP) organizes each virtual memory region into descending power-of-two logical chunks. A region of 13 base pages is represented as 8 + 4 + 1 pages, and for an address offset a in a region of size S the containing logical chunk is identified by MSB(a XOR S). Variable-size mappings already exist: BAP's candidate contribution is a compact region-relative representation and lookup rule, not the invention of large pages. In particular, a logical BAP chunk is not automatically a hardware-supported page mapping or one TLB entry.

This work formulates the logical scheme, provides a correctness argument for its lookup rule, and uses exploratory simulations to evaluate mapping-count tradeoffs, synthetic whole-chunk activation, permission-boundary refinement, and parameter sensitivity. A separate v5 audit tests the virtual-alignment and native-page-size eligibility of BAP's proposed chunks. No operating-system implementation or physical-memory translation mechanism is claimed.

The v6 extension completes virtual-to-physical address semantics in a controlled software model using explicit page-frame and permission metadata. It does not implement BAP in Linux, modify a hardware page walker, or measure TLB performance. The central open question is whether structured XOR/MSB parent lookup outweighs the descriptor, subrecord, and update costs of a generic range representation.

The v7 analysis now tests that cost question directly under a declared storage layout and comparable physical mappings. Its central outcome is negative for metadata space: generic ranges use fewer bytes in every tested case. BAP needs fewer child-list key comparisons on fragmented regions, but that metric does not account for the extra indexing stage or establish a latency improvement. These outcomes narrow the claim of this paper rather than imply a faster hardware translator.

The v8 pilot now tests full native-code software lookup cost against an explicit indexed-range competitor. It complements, but does not replace, the v7 metadata-model results or establish a hardware paging improvement.

Contributions

1. A region-relative logical chunk representation based on descending power-of-two decomposition.
2. A compact XOR/MSB lookup rule for identifying the logical chunk containing an address inside a binary-decomposed region.
3. A comparison of logical chunk counts for fixed, huge-page-style, idealized multi-size, aligned multi-size, and limited/unbounded BAP models under stated assumptions.
4. A workload-aware synthetic model of whole-chunk activation, split pressure, and permission-boundary refinement (not measured OS page faults).
5. A parameter sensitivity study and a v5 audit of virtual alignment and native leaf-page eligibility that identify conditions under which logical BAP chunks cannot be treated as conventional hardware translations.

6. A v6 software-model correctness check for physical-frame translation and protection, with explicit generic-range and native page-leaf comparators across fragmentation and alignment scenarios.

7. A v7 synthetic metadata-budget and instrumented lookup-comparison study, reporting both a negative storage result and the bounded interpretation of reduced child-list key comparisons.

8. A C++20 exploratory lookup microbenchmark (v8) against three generic-range implementations, including a region-relative indexed baseline; it reports workload-dependent rankings and cross-run variability rather than a claim of universal performance superiority.

2. Background and Motivation

2.1 Fixed-size paging

In fixed-size paging, a process address space is divided into equal-size virtual pages, and physical memory is divided into equal-size frames. For a base page size of 4 KB, the lower 12 address bits are the offset inside the page, and the remaining bits identify the virtual page number. This structure makes translation simple, but a large region requires one mapping entry per base page.

```
page_number = address >> 12
offset      = address & 0xFFF
```

2.2 Huge pages and superpages

Huge pages and superpages reduce the number of translations by allowing larger mappings. Prior work on superpages notes that one TLB entry can map a large physical memory region into a virtual address space, which increases TLB coverage and may reduce TLB misses [1]. Linux HugeTLB and Transparent Huge Pages are practical examples of large-page mechanisms. Linux documentation describes Transparent Huge Pages as a mechanism for automatically using huge pages as backing for virtual memory, including promotion and demotion of page sizes [2]. Linux HugeTLB documentation also highlights the importance of physically contiguous huge-page allocation, especially when huge pages are allocated early in boot [3].

Large pages can reduce translation-entry counts, but their feasibility is constrained by supported page sizes, alignment, backing, and protection requirements. Whether touching part of a larger range triggers whole-range allocation or loading is operating-system and policy dependent. Accordingly, the synthetic 'fault amplification' measure later in this paper is a modeled whole-chunk-activation ratio, not a universal prediction of hardware page faults.

2.3 Multi-size mappings and region-based designs

Modern processors and operating systems often support multiple page sizes. For example, x86-64 systems commonly support 4 KB, 2 MB, and 1 GB pages, and recent research such as Trident studies how to exploit all page sizes dynamically in Linux [5]. Region-based designs, including Direct Segments, also reduce translation overhead by mapping large contiguous regions using base, limit, and offset-style hardware support [4]. BAP is related to these ideas, but it is region-relative and binary-decomposed. It studies whether a compact bit-level lookup rule can organize variable-size chunks without treating the whole process as one fixed segment.

3. Binary Adaptive Paging Design

3.1 Design overview

Binary Adaptive Paging operates at the level of virtual memory regions rather than whole processes. A process normally contains several regions, such as code, read-only data, heap, mapped files, shared libraries, stack, and guard pages. These regions may have different permissions and backing policies, so the scheme decomposes each region independently. The smallest unit is still the normal base page; all sizes in this paper are measured in base-page units.

For a region of size S, the scheme represents S as a sum of descending powers of two. For example:

```
S = 13 = 8 + 4 + 1
S = 19 = 16 + 2 + 1
S = 10000 = 8192 + 1024 + 512 + 256 + 16
```

The chunks are laid out in descending order from the beginning of the region. For S = 13, the layout is [0,8), [8,12), and [12,13). Each chunk has a size that is a power of two, and the number of chunks is equal to the number of set bits in S if the unbounded scheme is used.

3.2 Limited and unbounded variants

The evaluation distinguishes two logical BAP variants. Unbounded BAP represents the region using the set-bit decomposition of S without an imposed maximum. Limited BAP divides any larger logical chunk into subchunks capped at 512 base pages in the main experiments. Although 512 base pages equals 2 MiB for a 4 KiB base page, these are logical region-relative chunks: the cap by itself does not establish native hardware support, virtual/physical alignment, or contiguity. The cap makes logical-size comparisons meaningful; hardware realizability is examined separately in Section 7.3.1.

Table 1. BAP variants used in the paper.

Variant	Meaning	Use in evaluation
binary_adaptive_limited	Caps region-relative logical chunks, e.g., at 512 base pages; not automatically native leaves	Size-capped logical comparison; virtual-alignment eligibility audited in v5
binary_adaptive_unbounded	Allows logical chunks larger than the common 512-base-page comparison cap	Theoretical upper bound

3.3 Adaptive split and merge motivation

A large logical chunk may be economical for a densely accessed uniform region, but a practical mapping design must handle sparse access, copy-on-write, permission boundaries, and updates. The v3 simulator approximates a policy that loads or activates an entire touched chunk and then counts segments produced by a simplified split rule. These figures describe pressure under that policy assumption; they are not measurements of real kernel splitting or hardware page-fault behavior.

4. Formal Model and Lookup Rule

4.1 Region model

A virtual memory region R is represented by a base address B, a size S measured in base pages, and attributes such as permissions and backing type. For a virtual address A inside the region, the normalized address offset is a = A - B. All formulas below use normalized offsets in base-page units.

```
R = (B, S, permissions, backing)
0 <= a < S
a = A - B
```

4.2 Binary decomposition

Let S be a positive integer. Its binary representation defines a descending list of powers of two. If bit k is set in S, then the decomposition contains a chunk of size 2^k. Chunks are ordered from larger powers to smaller powers.

```
decompose(S):
    chunks = []
    for k from floor(log2(S)) down to 0:
        if bit k of S is set:
            chunks.append(2^k)
    return chunks
```

4.3 XOR/MSB lookup rule

For an address offset a inside a region of size S, BAP computes x = a XOR S and then k = MSB(x), where MSB returns the index of the most significant set bit. The logical chunk size is then 2^k. The chunk base is obtained by clearing the lower k+1 bits of S:

```
x = a XOR S
k = MSB_position(x)
chunk_size = 1 << k
chunk_base = S & ~((1 << (k + 1)) - 1)
```

For example, when S = 13 = 1101b, the decomposition is 8 + 4 + 1. For a = 9, x = 9 XOR 13 = 4, so k = 2 and the address belongs to the 4-page chunk. For a = 12, x = 12 XOR 13 = 1, so k = 0 and the address belongs to the 1-page chunk.

4.4 Translation algorithm

```
locate_logical_chunk(A, region, max_chunk):
    B, S = region.base, region.size  # base-page units
    if A < B or A >= B + S:
        raise RegionMiss
    a = A - B
    k = MSB_position(a XOR S)
    logical_base = S & ~((1 << (k + 1)) - 1)
    logical_size = 1 << k
    if logical_size <= max_chunk:
        chunk_base, chunk_size = logical_base, logical_size
    else:
        sub_index = (a - logical_base) // max_chunk
        chunk_base = logical_base + sub_index * max_chunk
        chunk_size = max_chunk
    return chunk_base, chunk_size, a - chunk_base
# Logical chunk only; physical mapping handled separately.
```

The algorithm above locates a logical chunk and an intra-chunk offset; it is not a complete virtual-to-physical translation algorithm. Such a translation additionally needs a region-discovery mechanism, a mapping from each eligible chunk or base page to physical backing, appropriate permissions, and a defined TLB/page-walker interface. Section 7.3.1 tests the necessary virtual prerequisites under selected page-size profiles, while physical prerequisites remain open.

4.5 Physical-frame and permission semantics (v6)

A virtual-to-physical translation requires more than the XOR/MSB parent index. In v6, a region descriptor provides its virtual start and length; a reference map associates each base page with a physical frame number (PFN) and read/write/execute rights. The BAP model first finds the 512-page-capped logical parent using XOR/MSB. Within that parent, it searches subrecords that contain physically consecutive frames and uniform permissions. A selected record yields physical address PA = (start_PFN + (VPN - record_start_VPN)) × 4096 + byte_offset, subject to bounds and permission checks.

The comparator range models encode identical physical and permission runs without BAP parent boundaries: one limits each descriptor to 512 pages and the other allows arbitrary run lengths. An x86-like leaf model permits 4 KiB leaves or 2 MiB leaves only when the starting virtual page and PFN are both multiples of 512, all 512 PFNs are consecutive, and their permissions agree. This construction validates software-model translation semantics; it assumes a hypothetical region-aware lookup path for BAP and generic ranges. It does not establish native hardware support or a TLB-fill mechanism.

4.6 Capped-parent indexing and metadata contract (v7)

The v7 program locates the capped BAP parent with arithmetic in the region-relative coordinate system. Let C = 512 base pages and S be the region length; set full = floor(S/C). For an offset a < full*C, the parent index is floor(a/C). Otherwise let rem = S mod C and k = MSB((a - full*C) XOR rem); the parent index is full + popcount(rem >> (k+1)). The index selects a parent-directory entry containing the start and count of child-run records. The record within that parent is found by binary search, after which the same physical-frame and permission semantics from Section 4.5 apply.

The arithmetic index avoids scanning all parent boundaries, but it adds work beyond the measured child-list comparisons: bit operations, arithmetic, the parent-directory read and bounds/protection checks. It also assumes that the appropriate region descriptor has already been found. Correctness is tested against explicit parent lists for region sizes 1–1,099 and starts 0, 1 and 7; this does not establish latency or TLB integration.

5. Correctness Argument

The central correctness property is that MSB(a XOR S) identifies the power-of-two chunk containing a when the region is decomposed into descending powers of two. The proof is based on the fact that the descending binary chunks correspond to intervals formed by clearing suffixes of S.

Theorem

Let S be a positive integer and let 0 <= a < S. Suppose S is decomposed into descending powers of two according to its set bits, and the resulting chunks are laid out from lower to higher offsets. Let k = MSB(a XOR S). Then a lies in the chunk of size 2^k whose base is S with all bits below and including k cleared.

Proof sketch

Because a < S, the binary strings of a and S must differ at some most significant bit. Let k be the highest bit at which they differ. Since S is larger than a, S has bit k equal to 1 and a has bit k equal to 0 after the common higher prefix. Bits above k are equal in both numbers. Therefore, clearing the lower k+1 bits of S gives the start of the interval whose length is 2^k. The address a shares the higher prefix, has 0 at bit k, and can vary in lower bits, so it falls inside that interval. This interval is exactly the chunk corresponding to bit k in the descending binary decomposition. Since XOR marks exactly the bit positions where a and S differ, MSB(a XOR S) returns this same k.

The simulator also validates this rule exhaustively for all region sizes up to the configured maximum by comparing the XOR/MSB lookup with a naive boundary scan. This validation is not a formal proof by itself, but it is useful as an implementation sanity check.

6. Simulation Methodology

6.1 Goals

The evaluation consists of two distinct types of evidence. Simulators v2–v4 estimate logical region chunk counts, an optimistic TLB-reach proxy based on choosing the largest chunks, and synthetic whole-chunk activation and splitting behavior. Simulator v5 checks virtual alignment and native page-size eligibility of the same logical chunks for two explicit hypothetical/conventional leaf-size profiles. Neither class of experiment measures real TLB hit rates, page-walk latency, physical contiguity, or end-to-end program performance.

6.2 Baselines

Table 2. Mapping strategies evaluated by the simulator.

Strategy	Description	Purpose
fixed	One mapping per base page	Standard page-based baseline
huge_mixed	512-page mappings plus base-page remainder	Simple huge-page-style baseline
multi_size_unaligned	Greedy use of multiple sizes without alignment constraints	Strong idealized baseline
multi_size_aligned	Greedy multi-size mapping with virtual alignment constraints	More realistic multi-size baseline
binary_adaptive_limited	BAP with maximum chunk cap	Fair BAP comparison

Strategy	Description	Purpose
binary_adaptive_unbounded	BAP without maximum chunk cap	Theoretical upper bound

6.3 Metrics

Table 3. Main simulation metrics.

Metric	Meaning
logical_mapping_count	Number of region-relative logical chunks (not necessarily hardware leaf entries)
estimated_tlb_reach_pages	Optimistic largest-chunk coverage proxy; not simulated TLB hit rate
alignment_penalty_entries	Difference in logical counts when absolute virtual alignment is imposed
fault_amplification	Modeled loaded/touched ratio under whole-chunk-activation assumption
adaptive_split_added_entries	Extra entries introduced by adaptive split
permission_refined_mapping_count	Mapping count after splitting at permission boundaries

6.4 Workloads

Table 4. Synthetic workloads in Simulator v3.

Workload	Description
dense_sequential	Every page in the region is touched
sparse_random	About 2% of pages are touched randomly
very_sparse_random	About 0.2% of pages are touched randomly
hot_cold_clustered	One hot cluster plus a small number of cold random touches
boundary_sparse	Pages near chunk boundaries are touched
cow_burst	A small contiguous write burst approximates COW pressure
mixed_permissions	Synthetic permission boundaries force mapping refinement

6.5 Experimental settings

Unless stated otherwise, one base page is the simulation unit and the principal cap is 512 base pages. The v2–v4 experiments use a 64-slot optimistic largest-chunk coverage proxy (not a TLB simulator); structural region sizes include 1–10,000 pages, and workload-aware runs focus on 1,000, 4,096, and 10,000 pages. Sensitivity runs vary BAP's cap among 128, 256, 512, 1,024, and 2,048 pages and the density threshold among 0.05, 0.10, 0.25, and 0.50,

over five seeds. In v5, the cap is 512 pages, region sizes are 13, 19, 513, 1,000, 4,096, and 10,000, and starts are 0, 1, 7, 13, 64, and 255, for two leaf-size profiles (72 cases). v5 tests only virtual alignment and page-size membership, not physical backing or actual TLB behavior.

6.6 v6 translation-correctness protocol

The v6 experiment uses a 4 KiB base page and considers three region sizes (1,000, 4,096, and 10,000 pages), three starting VPNs (0, 1, and 7), and five deterministic backing layouts: contiguous, contiguous_phase_matched, fragmented_32, mixed_permissions, and fragmented_permissions. The Cartesian product contains 45 cases. Each strategy receives the same page-to-PFN map and permissions. Every base page is checked at byte offsets 0, 17, and 4,095 for read, write, and execute access, including expected protection faults; out-of-region accesses are also rejected by validation. The code reports structural parent, subrecord, range, and x86-like leaf counts, not allocation costs or runtime metrics.

In the contiguous layout, physical frames begin at an aligned PFN independently of the region start, which can mismatch the virtual 2 MiB alignment phase. The contiguous_phase_matched layout aligns the virtual and physical phases; fragmented_32 adds a deterministic discontinuity every 32 pages. Mixed-permission cases divide the region into RX, R, and RW thirds. Validation was reproduced using python translation_v6.py --validate, with 45 rows identical to the supplied v6 CSV. The numerical comparison refers exclusively to the specified software model and its assumptions.

6.7 v7 modeled metadata and lookup-comparison protocol

v7 reuses the same per-base-page PFN and permission reference from v6 and the same three representations: capped BAP parents with child runs, generic uniform-permission contiguous ranges capped at 512 pages, and unbounded contiguous ranges. Native x86 page-table storage is not included because a fair byte comparison would require a separate page-table allocation model. The representation study assigns 32 bytes to a region descriptor for all models, 24 bytes to each permission-uniform physical run record for all models, and an additional 8 bytes for each BAP parent-directory slot. These sizes are declared packing assumptions, not measured memory usage.

The resulting budget is BAP_bytes = 32 + 24*BAP_child_records + 8*BAP_parents; capped_range_bytes = 32 + 24*capped_records; and unbounded_range_bytes = 32 + 24*unbounded_records. Region discovery and its index, allocator overhead, page-table pages, cache-line occupation, shootdowns, and physical TLB storage are outside this byte model. Comparisons count only binary-search tests on the start-VPN key of a child/run record; BAP XOR/MSB and parent indexing are not charged as comparisons.

The factorial design includes region lengths 1,000, 4,096 and 10,000 pages; starting VPNs 0, 1 and 7; five backing/permission layouts; uniform-random, sequential, hot/cold and boundary-stress address patterns; and seeds 1, 2 and 3. Thus, 3*3*5*4*3 = 540 scenario/trace rows are recorded, with 1,024 lookups per row by default. Every modeled translation is checked against the reference PFN and permissions. Results include mean, 95th-percentile and maximum search-key comparisons; the next section reports mean values for three representative layouts with uniform-random accesses.

Table 4a. Assumed compact layout for v7 (abstract software representations).

Object	Bytes	Coverage / qualification
Region descriptor	32	Charged to all representations; descriptor discovery excluded
Physical contiguous-run record	24	Same run-record structure for BAP and generic ranges
BAP parent-directory slot	8	Child start-index + child count; additional BAP cost

Note. The run record comprises start VPN (8 bytes), length (4), start PFN (8), permissions (1), and padding/reserved bytes (3). A parent slot uses two 4-byte indices. Byte totals are layout assumptions, not Python object sizes or real kernel allocations.

6.8 Native-code software lookup microbenchmark protocol (v8)

To test whether reduced child-list comparisons translate into lower software lookup time, v8 implements four translation-record search variants in a compiled C++20 microbenchmark: (1) BAP arithmetic parent selection plus a local binary search of child runs; (2) a global binary search of capped (512-page) contiguous-range records; (3) a global binary search of unbounded contiguous-range records; and (4) a stronger indexed-range scheme using the same capped-range records but an 8-byte directory entry per 512-page region-relative bucket, followed by a bucket-local binary search. Each method returns the same VPN-to-PFN and protection information. All cases undergo exhaustive page-wise semantic validation before timing; the timed path incorporates a translation-dependent checksum to discourage unused-result elimination.

The test grid includes region sizes 1,000, 4,096 and 10,000 pages; initial VPNs 0, 1 and 7; five deterministic physical/permission layouts; four address-access patterns (uniform random, sequential, hot/cold and boundary stress); and trace seeds 1, 2 and 3, totaling 540 case/trace rows per program execution. Each row is timed using 4,096 lookups and five repeated measurement passes with rotated method order. The per-method median ns/lookup is recorded. Two separate executions of the supplied pilot on a virtualized x86_64 host yielded 540 rows each. Boundary-stress traces are deterministic and the nominal seeds are not independent samples for that pattern. These are *in-process software-record lookups*, not kernel page-table translation or TLB timing.

The pilot host was a virtualized Intel Xeon Platinum 8573C system (KVM, five allocated vCPUs), using GCC 14.2.0 with -O3 -std=c++20. Metadata byte estimates retain v7 assumptions: a shared 32-byte region descriptor, 24 bytes per run record, and 8 bytes per BAP-parent or indexed-range directory slot. They exclude container/allocator overhead and region-discovery indexes. Timing excludes mapping creation and updates, OS page walks, TLB activity, shootdowns, and whole-application behavior. Microsecond and nanosecond-scale numbers from virtualized execution must not be generalized beyond this pilot.

Table 4b. Lookup schemes and what is actually timed (v8).

Scheme	Search interface	Extra directory
BAP	Parent arithmetic + parent-local binary search	8 B per BAP parent
Range cap512	Global binary search in capped run list	None
Range unbounded	Global binary search in unrestricted run list	None
Range indexed	512-page bucket directory + local binary search	8 B per bucket

Note. Every variant is a software metadata lookup over synthetic layouts, not a supported x86 leaf mapping. The common region descriptor is treated as already selected; discovery among many VMAs is excluded.

7. Results

7.1 Structural mapping-count results

The original structural experiment compares logical chunk counts when regions start at page 0. The BAP limited and idealized multi-size strategies share the same maximum logical block size. This controls the logical decomposition comparison, but does not establish equivalent native page-table support or TLB representation.

Table 5. Fair structural comparison for aligned region start.

Region size	Fixed	Huge mixed	Multi-size unaligned	Multi-size aligned	BAP limited	BAP reduction vs fixed (%)
13	13	13	3	3	3	76.92
19	19	19	3	3	3	84.21

Region size	Fixed	Huge mixed	Multi-size unaligned	Multi-size aligned	BAP limited	BAP reduction vs fixed (%)
512	512	1	1	1	1	99.80
513	513	2	2	2	2	99.61
1000	1000	489	6	6	6	99.40
4096	4096	8	8	8	8	99.80
10000	10000	291	21	21	21	99.79

Note. Counts are logical chunk counts; a 512-page cap alone does not imply that every resulting block is a native 2 MiB translation.

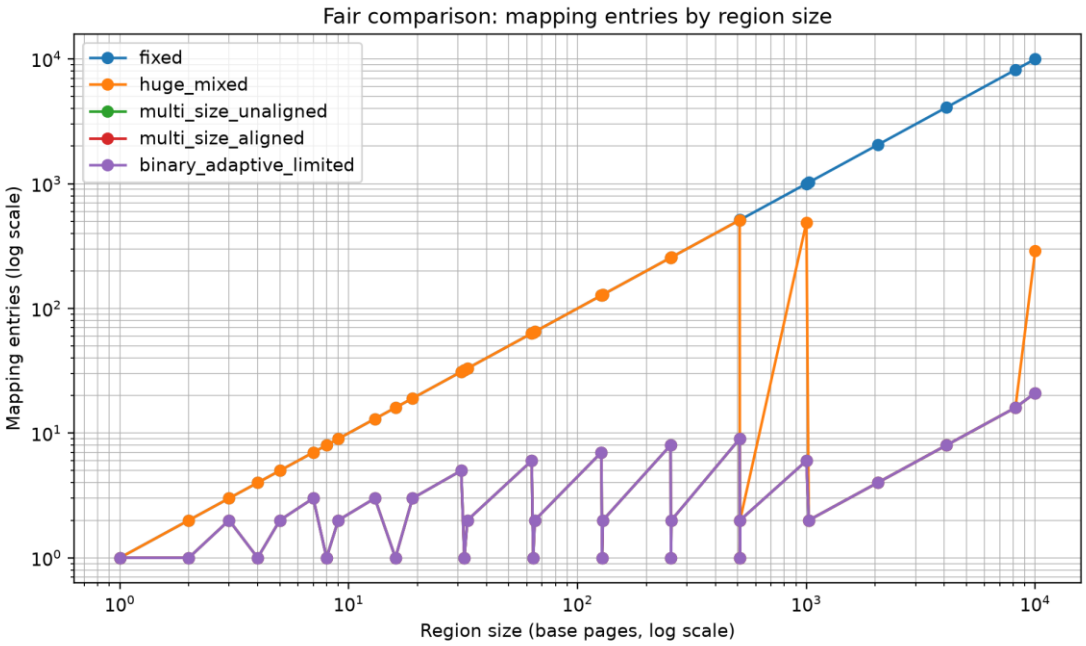


Figure 1. Fair comparison: mapping entries by region size.

In the logical decomposition model, a 10,000-page region uses 10,000 fixed-size chunks or 21 BAP limited chunks. For aligned starts and the same maximum chunk size, BAP limited matches the strong idealized multi-size models rather than improving their count. These counts should not be interpreted as actual page-table entries or hardware TLB entries without an implementation model and page-size/alignment eligibility checks.

7.2 Upper-bound behavior

Table 6. Upper-bound comparison with unbounded BAP.

Region size	Fixed	Huge mixed	BAP limited	BAP unbounded	Unbounded reduction vs fixed (%)
13	13	13	3	3	76.92

Region size	Fixed	Huge mixed	BAP limited	BAP unbounded	Unbounded reduction vs fixed (%)
19	19	19	3	3	84.21
512	512	1	1	1	99.80
513	513	2	2	2	99.61
1000	1000	489	6	6	99.40
4096	4096	8	8	1	99.98
10000	10000	291	21	5	99.95

Note. Unbounded BAP is a logical decomposition upper bound, not a claim of support for an arbitrary hardware page size.

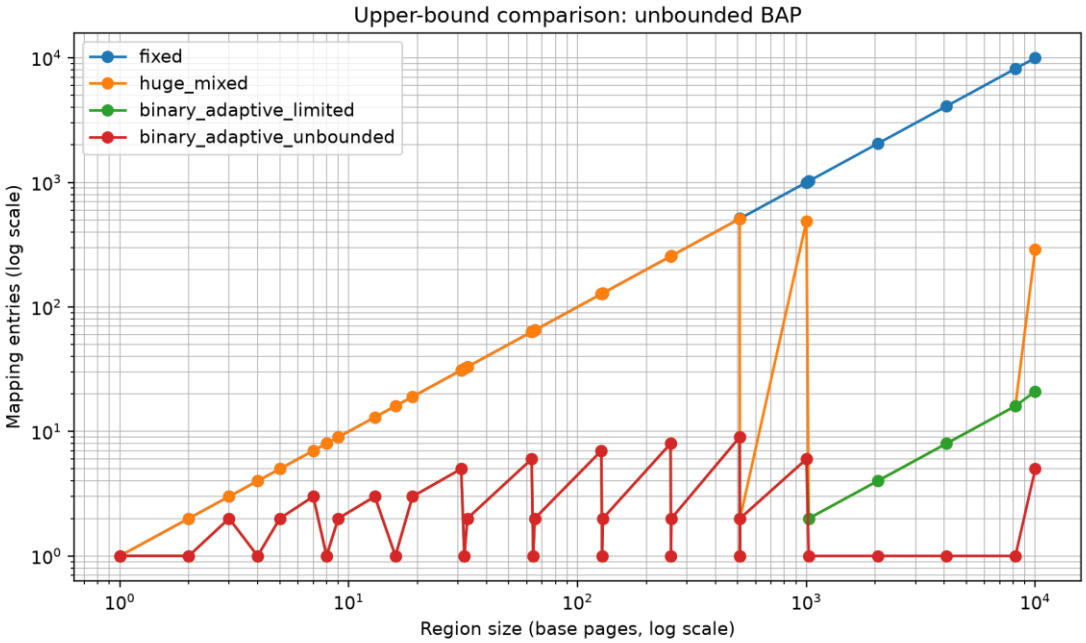


Figure 2. Upper-bound comparison: unbounded BAP.

Unbounded BAP decomposes 4,096 pages into one logical chunk and 10,000 pages into five logical chunks (8,192 + 1,024 + 512 + 256 + 16). This is an upper bound on logical representation size only; it does not assume existing hardware can implement each chunk as an independent translation leaf.

7.3 Alignment sensitivity

Table 7. Logical chunk counts under region-start offsets (abstract v2 model).

Region start	Multi-size unaligned	Multi-size aligned	BAP limited	BAP unbounded	Alignment penalty
0	21	21	21	5	0
1	21	30	21	5	9

Region start	Multi-size unaligned	Multi-size aligned	BAP limited	BAP unbounded	Alignment penalty
7	21	30	21	5	9
13	21	30	21	5	9
64	21	24	21	5	3
255	21	25	21	5	4

Note. BAP chunks are relative to the region base and are NOT necessarily legal native hardware page mappings. Section 7.3.1 audits the prerequisite alignment and size support.

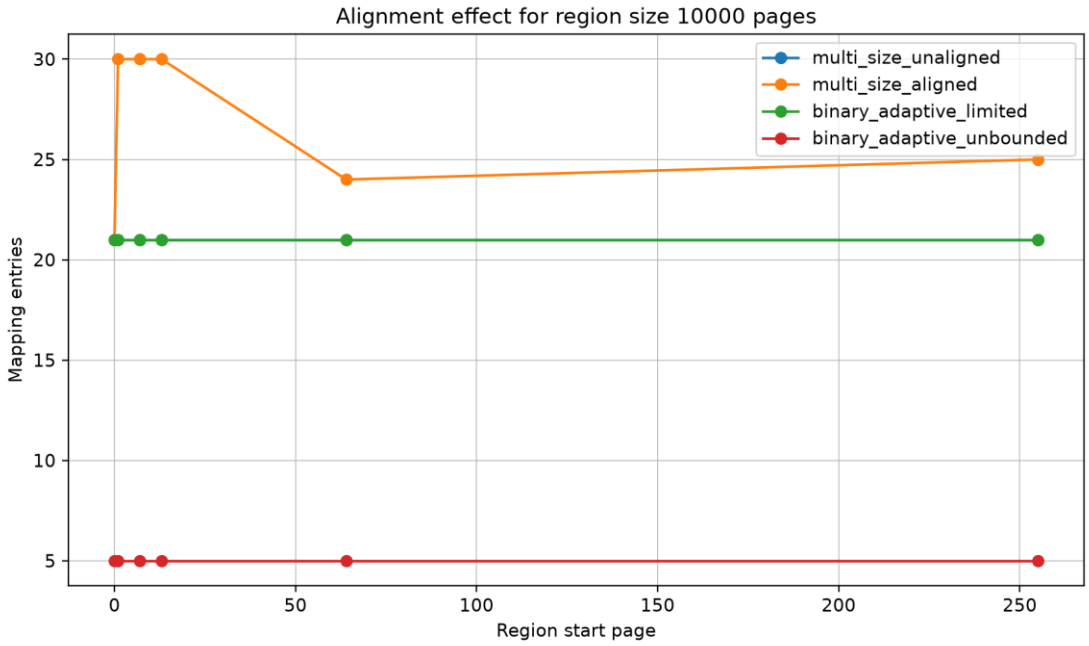


Figure 3. Logical region-relative chunk counts versus aligned multi-size counts (not TLB results).

The original v2 model counts 21 BAP limited logical chunks for a 10,000-page region regardless of whether the region starts at page 0, 1, 7, or 13; the alignment-constrained multi-size model counts 21 for start 0 and 30 for starts 1, 7, or 13. This demonstrates an assumption of the region-relative representation, not 21 immediately realizable hardware translation leaves. A native large-page entry requires an allowed size and virtual alignment, with further physical and protection constraints. The feasibility check below tests these necessary virtual conditions rather than assuming them away.

7.3.1 Virtual-alignment and page-size feasibility (v5)

The v5 structural audit examines whether each original BAP limited block has an allowed native leaf size and begins at an absolutely aligned virtual address. It reports (i) original logical blocks, (ii) blocks individually eligible for a native leaf, (iii) legal aligned leaves obtained by splitting within original BAP block boundaries, and (iv) leaves obtained by realigning/retiling the entire region. Two explicit profiles are considered: 'ideal powers of two' (every native size 1, 2, 4, ..., 512 base pages, hypothetical) and 'x86 4 KiB/2 MiB' (native leaf sizes 1 and 512 base pages only; 1 GiB leaves are excluded by this 512-page cap). All counts assume adequate physical backing and do not model TLB operation.

Table 7a. Virtual eligibility and aligned retiling, S = 10,000 base pages (v5).

Profile	Start page	BAP logical chunks	Directly eligible chunks	Split only: legal leaves	Retile: aligned leaves
Ideal powers of two	0	21	21	21	21
Ideal powers of two	1	21	0	204	30
Ideal powers of two	64	21	1	80	24
x86 4 KiB/2 MiB	0	21	19	291	291
x86 4 KiB/2 MiB	1	21	0	10000	802
x86 4 KiB/2 MiB	64	21	0	10000	802

Note. Direct eligibility tests only allowed size and virtual alignment, not physical feasibility. Split-only keeps BAP boundaries and is not optimized. Retiling may change them. The all-powers-of-two profile is hypothetical.

Interpretation. The apparent 21-versus-30 advantage in Table 7 is a logical representation comparison, not a native hardware paging advantage. Under the hypothetical all-powers-of-two profile, legal retiling at start page 1 needs the same 30 aligned leaves as the aligned multi-size baseline. Under the limited x86 4 KiB/2 MiB profile, it needs 802 legal leaves; the original 21 BAP chunks cannot serve as native page-table leaves. These findings do not rule out a newly designed region-relative translation mechanism, but such a mechanism must specify its region lookup, physical mapping, metadata costs, protection semantics, and TLB interface. No end-to-end speedup is implied.

7.3.2 End-to-end translation semantics and physical-layout checks (v6)

Unlike v5, which checked necessary virtual leaf prerequisites, v6 builds byte-address translations from specific physical frame sequences and permission maps. BAP uses a two-stage software lookup: a region-relative XOR/MSB parent plus a search among contiguous, permission-uniform child subrecords. Two generic range representations and an x86-like page-leaf construction act as comparators. All four models must return the same physical address for permitted accesses and reject disallowed accesses. A region-aware hardware translation interface for BAP is assumed, not demonstrated.

Table 7b. Structural translation representation counts under five backing layouts (v6; region S = 10,000 pages, starting VPN = 1).

Physical backing	BAP parents	BAP subrecords	Generic range ≤512	Generic range unlimited	x86-like leaves
Contiguous; phase differs	21	21	20	1	10000
Contiguous; phase matched	21	21	20	1	802
Fragmented every 32 pages	21	313	313	313	10000
Three permission zones	21	23	21	3	10000
Fragmented + permissions	21	315	315	315	10000

Note. BAP subrecords exclude parent-index and region-descriptor storage; range records are not TLB entries. The x86-like leaf count reflects eligibility under virtual/physical alignment and uniform-permission constraints only. Generic ranges and BAP use hypothetical lookup support. No memory-byte or time cost is measured.

With contiguous uniform backing, 21 BAP parent chunks correspond to 21 physical/permission subrecords, while a capped generic-range partition needs 20 records and an unbounded range needs only one. Therefore, the structural results do not show that BAP is inherently more compact than generic range metadata. When physical continuity is broken every 32 pages, both BAP and generic range representations require 313 subrecords or records. Under three protection zones, BAP needs 23 subrecords versus 21 capped or 3 unbounded generic-range records. The parent boundaries do not eliminate extra fragmentation and protection metadata.

The x86-like outcome depends sharply on *both* virtual and physical alignment. At starting VPN 1, contiguous frames beginning at a 2 MiB-aligned PFN have a different phase from the virtual start; no 2 MiB leaf is eligible, so the model needs 10,000 base-page leaves. When phases are matched, 18 2 MiB leaves become eligible and the total falls to 802 leaves. The latter matches the best-case virtual-alignment count from v5, but only under the added physical assumptions. Across all 45 scenarios, translation and permission validation succeeded in the local

reference-model rerun; no TLB latency, metadata byte budget, allocation dynamics, or operating-system integration was evaluated.

7.4 Workload-aware fault amplification

The workload-aware v3 simulation models a deliberate whole-chunk-activation rule: if a chunk is touched, all base pages in that chunk are counted as loaded until a split rule is applied. The ratio (modeled loaded pages)/(unique touched pages) is labeled 'fault amplification' in the original plots. It is not a measured OS page-fault rate, nor a necessary consequence of an abstract translation chunk. In this synthetic model, dense access gives a ratio of 1.0.

Table 8. Selected workload-aware stress results for a 10,000-page region.

Strategy	Workload	Mappings	Touched	Loaded no split	Amplification no split	Adaptive mappings	Loaded after split	Amplification after split
fixed	sparse_random	10000	200	200	1	10000	200	1
binary_adaptive_limited	sparse_random	21	200	9984	49.92	416	200	1
binary_adaptive_unbounded	sparse_random	5	200	9984	49.92	402	200	1
binary_adaptive_limited	hot_cold_clustered	21	550	9216	16.76	113	1070	1.95
binary_adaptive_unbounded	hot_cold_clustered	5	550	9728	17.69	606	550	1
fixed	cow_burst	10000	100	100	1	10000	100	1
binary_adaptive_limited	cow_burst	21	100	512	5.12	122	100	1
binary_adaptive_unbounded	cow_burst	5	100	8192	81.92	106	100	1

Note. All loaded-page/amplification results are from the synthetic whole-chunk-activation assumption; split costs, actual allocation behavior, and TLB invalidations are not modeled.

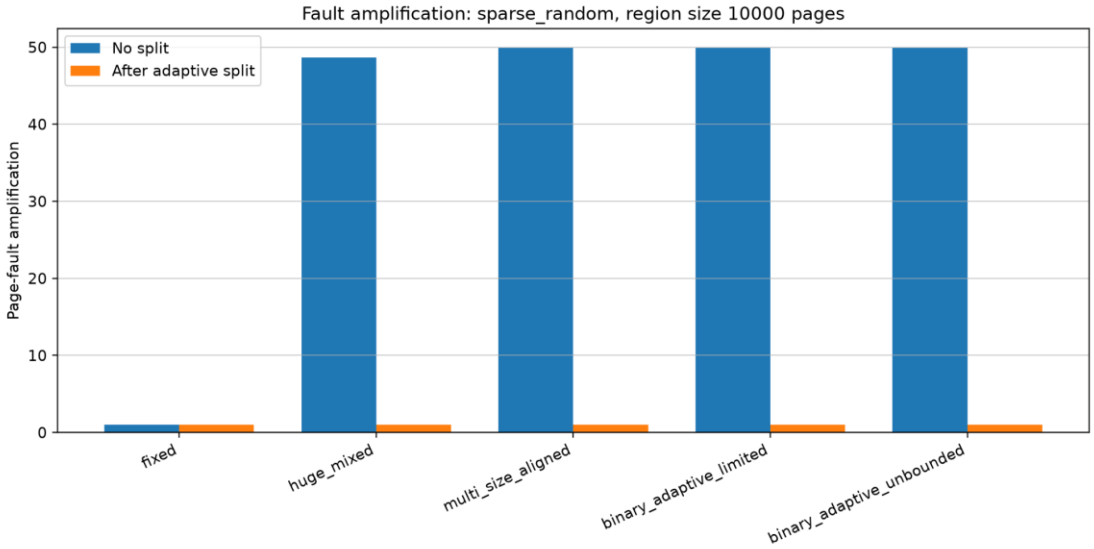


Figure 4. Fault amplification under sparse random access.

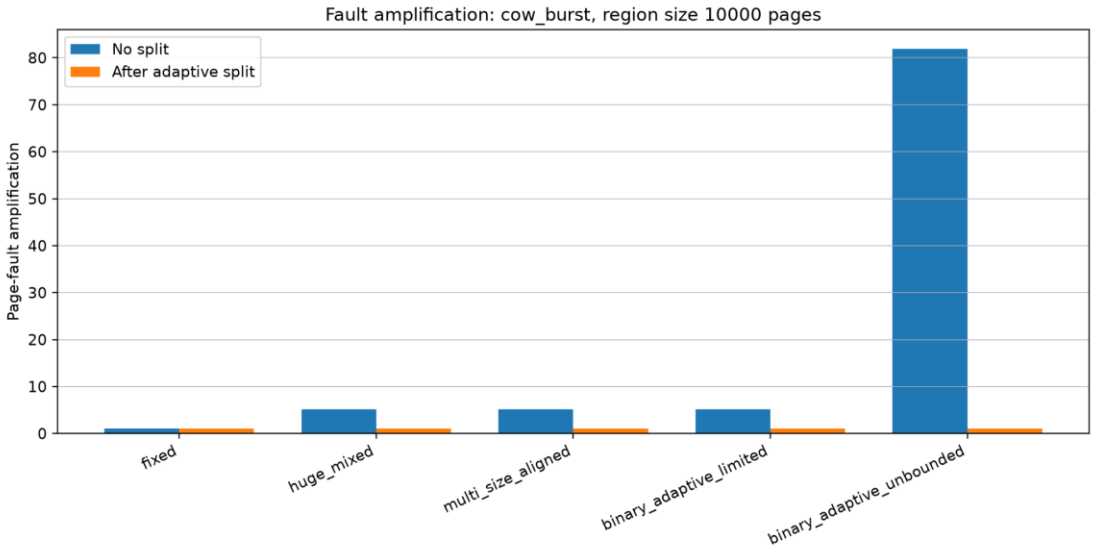


Figure 5. Fault amplification under COW-like burst access.

For the 10,000-page synthetic region, the sparse workload touches 200 distinct pages. The whole-chunk-activation model counts 9,984 pages before splitting for BAP limited, giving a modeled ratio of 49.92; the fixed-page model counts 200 pages (ratio 1.0). For the COW-like burst, the modeled BAP limited ratio is 5.12 and unbounded BAP is 81.92. These observations identify a sensitivity of the chosen activation policy to chunk size. They do not independently establish page-fault behavior or a special BAP advantage over conventional aligned multi-size mappings.



Figure 6. Loaded pages by workload for BAP limited.

7.5 Permission-boundary refinement

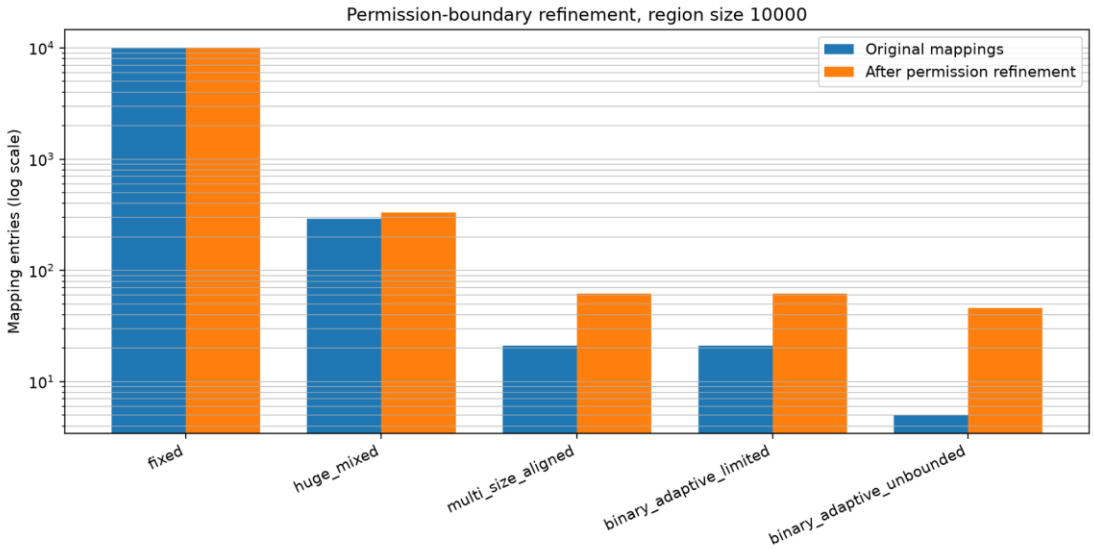


Figure 7. Mapping refinement after synthetic permission boundaries.

Under synthetic permission-boundary refinement, the 10,000-page example increases BAP limited from 21 to 62 logical segments and unbounded BAP from 5 to 46. This models segmentation pressure, not a validated hardware page-table construction; the resulting segments must still satisfy page-size, virtual/physical alignment, and protection requirements for native mappings.

7.6 Sensitivity analysis

Table 9. Mapping-count sensitivity for BAP limited.

Max BAP chunk	Mean mappings
128	79
256	40
512	21
1024	12
2048	8

Note. Increasing the logical BAP cap reduces chunk count in this sweep; the comparator cap was not co-varied, so cross-strategy trends at different caps are not a controlled native-hardware comparison.

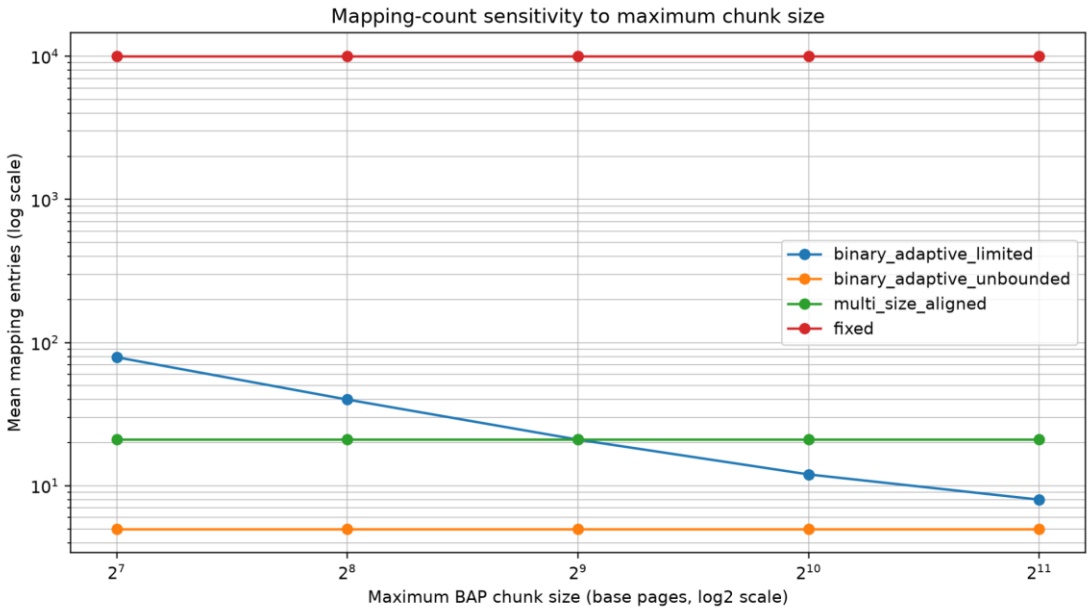


Figure 8. Mapping-count sensitivity to maximum chunk size.

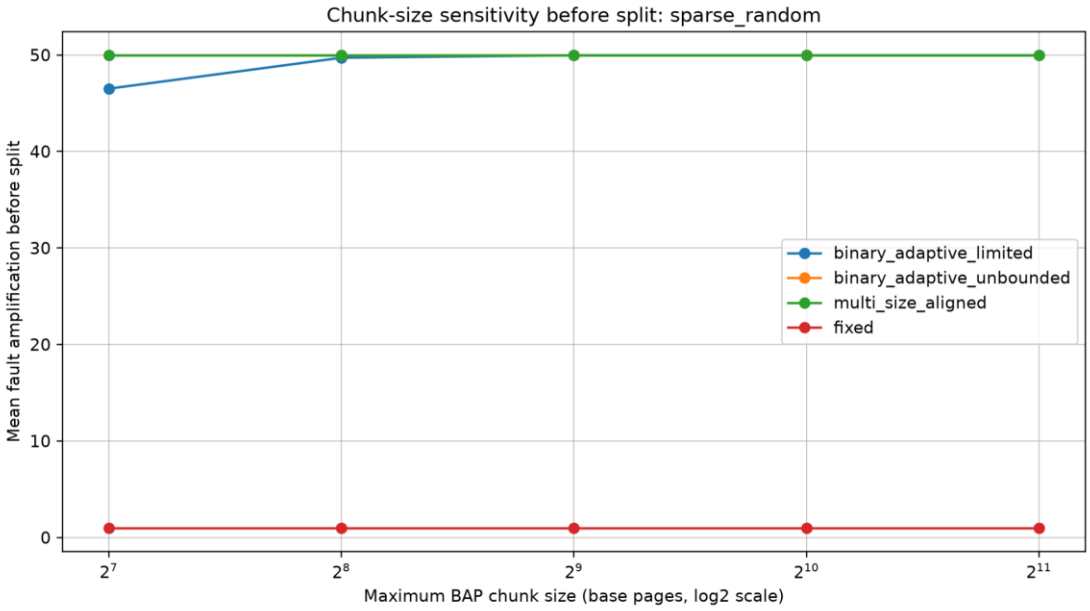


Figure 9. Chunk-size sensitivity before split under sparse random access.

Table 10. Split-threshold tradeoff for BAP limited under hot/cold workload.

Split threshold	Mean amplification after split	Mean added entries
0.05	1.95	93
0.10	1.95	93
0.25	1.95	93
0.50	1	599.40

Note. *Higher split thresholds reduce amplification but can add many mappings.*

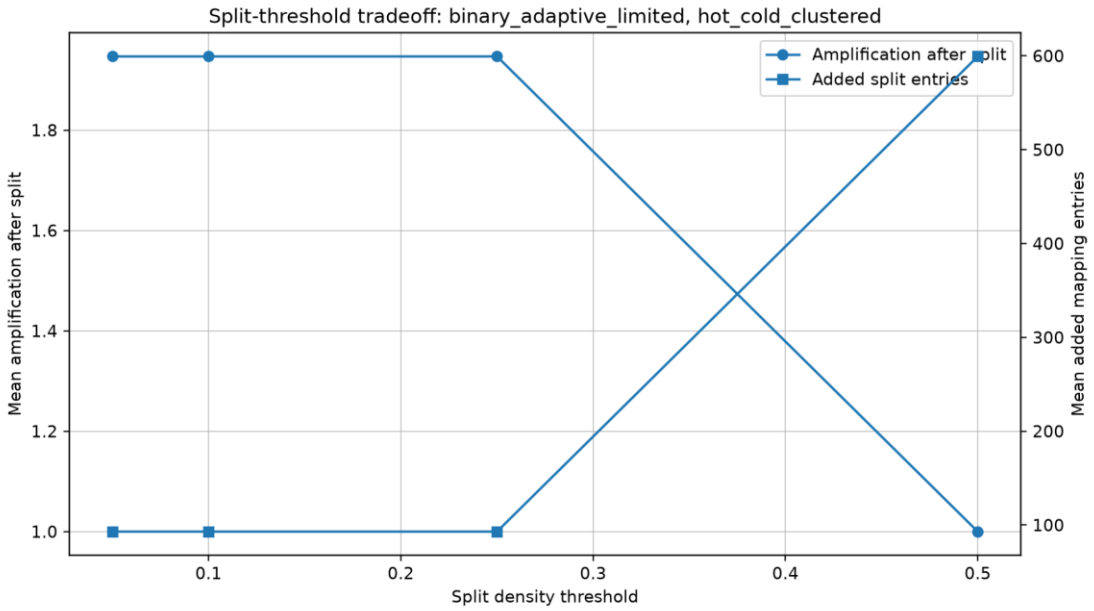


Figure 10. Split-threshold tradeoff for BAP limited.

Sensitivity runs show that increasing the logical BAP cap from 128 to 2,048 pages reduces its chunk count from about 79 to 8 for the 10,000-page region. More aggressive synthetic splitting can reduce the modeled whole-chunk-activation ratio, but at the expense of extra segments. These trends are within-model parameter effects; native leaf-size eligibility, physical backing, and runtime costs were not included. In particular, Figure 8 varies BAP's cap while the comparison baseline's cap is fixed, so it should not be read as a fair hardware mapping advantage.

7.7 Metadata-budget and key-comparison results (v7)

For comparability with the v6 examples, the tables below select a 10,000-page region starting at VPN 1. All byte totals are derived from the declared v7 model; they are invariant across access patterns and random seeds for a given physical/permission layout. The search-key means are averaged across three seeds of the uniform-random pattern (1,024 lookups per seed).

Table 11. Modeled metadata bytes at S = 10,000, starting VPN = 1 (v7).

Physical / permission layout	BAP parents / child records	BAP bytes	Range <=512 bytes	Range unbounded bytes
Contiguous	21 / 21	704	512	56
Fragmented every 32 pages	21 / 313	7,712	7,544	7,544
Three permission zones	21 / 23	752	536	104
Fragmented + permissions	21 / 315	7,760	7,592	7,592

Note. All three representations include the same 32-byte descriptor; BAP additionally carries one 8-byte slot per parent. Bytes are modeled assumptions, not hardware or process-memory measurements.

In the 540 reported rows, BAP never has a lower model-byte total than either generic-range representation. With contiguous backing, BAP uses 704 bytes, capped ranges 512, and unbounded ranges 56; with 32-page physical fragmentation the respective amounts are 7,712, 7,544 and 7,544 bytes. Thus, v7 provides no support for an unconditional BAP metadata-compression claim under the tested layout. Different encodings remain an open design question and cannot be credited without implementation.

Table 12. Mean run-start binary-search key comparisons (uniform random, three seeds; v7).

Physical / permission layout	BAP child-list	Range <=512 global	Range unbounded global
Contiguous	1.000	4.464	1.000
Fragmented every 32 pages	4.039	8.377	8.377
Three permission zones	1.024	4.512	2.000

Note. Counted operations are search-key comparisons only. BAP parent selection, XOR/MSB, popcount, integer arithmetic, parent-directory loads, and all cache or memory-latency effects are excluded.

For fragmented backing, BAP averages 4.039 child-list key comparisons versus 8.377 global range comparisons in both range baselines. With a single contiguous range, BAP and the unbounded-range baseline each use one record-key comparison, while BAP additionally needs a parent lookup. The instrumented reduction in key comparisons does not establish a lower complete instruction count, fewer cache misses, lower translation latency, or better performance on actual processors.

7.8 Exploratory software lookup latency and indexed-range baseline (v8)

The following descriptive tables summarize the supplied v8 pilot results only. For each layout, S = 10,000 base pages, initial VPN = 1, and uniform-random access, the figures are medians across three trace seeds in each of two separate program executions (six case-level observations). The complete dataset consists of two 540-row CSV files. The table is not a latency claim for a production memory-management system.

Table 13. Pilot software lookup ns/access (S = 10,000; initial VPN = 1; uniform random; v8).

Physical / permission layout	BAP	Range cap512	Range unbounded	Range indexed
Contiguous	3.36	24.43	3.50	2.33
Contiguous, phase matched	3.36	24.42	3.36	2.33
Fragmented every 32 pages	27.66	51.50	56.45	31.53
Fragmented + permissions	29.35	50.76	57.32	29.28
Mixed permissions	3.76	24.85	10.30	5.46

Note. Values are descriptive case medians from two independent pilot program executions on a virtualized host. They are not confidence bounds, p-values, hardware translation latencies, or measurements from the user's local CSV.

The ranked outcome changes by layout. Indexed ranges were faster with contiguous backing (2.33 vs. 3.36 ns/access for BAP); BAP had a lower reported pilot median under 32-page fragmentation (27.66 vs. 31.53 ns/access); and the two were close under combined fragmentation and permission changes (29.35 vs. 29.28 ns/access). BAP generally outpaced a naive global binary search of many records, but such a comparison alone is not adequate evidence of novelty or overall superiority. The indexed-range control is essential because it can exploit the same locality used by BAP parent search.

Table 14. Hypothetical model metadata bytes for v8 compared schemes (same scenario subset).

Physical / permission layout	BAP (B)	Range cap512 (B)	Range unbounded (B)	Range indexed (B)
Contiguous	704	512	56	672
Contiguous, phase matched	704	512	56	672
Fragmented every 32 pages	7,712	7,544	7,544	7,704
Fragmented + permissions	7,760	7,592	7,592	7,752
Mixed permissions	752	536	104	696

Note. Bytes are assumed packed representation sizes, not measured process RSS, allocator footprint, CPU-cache traffic, or actual page-table memory. The indexed control adds an 8-byte slot per 512-page bucket.

The modeled storage cost does not favor BAP in this illustrative subset. For contiguous backing it requires 704 bytes against 672 for indexed ranges, 512 for globally searched capped ranges and 56 for unbounded ranges; with 32-page fragmentation the BAP/indexed totals are 7,712/7,704 bytes. The pilot therefore offers no evidence that BAP is more compact than a comparable indexed contiguous-range organization.

Repeatability is not yet adequate for publication-grade performance claims. The research note reports a roughly 24% 95th-percentile absolute relative change across the two pilot runs for BAP and indexed ranges. No hardware counters or confidence intervals from controlled independent physical-host trials were collected. Results should be kept explicitly exploratory until replicated under pinned-CPU conditions and compared using multiple independent processes and traces.

7.8.1 Independent local validation status

A separate user-executed build of lookup_benchmark_v8 completed with the exact console summary "PASS: validated every address and permission for all layouts. 540 timing rows; checksum=176" when run with --out results_v8/local_run.csv --queries 4096 --repeats 5. This establishes that the local run completed its implemented validation and wrote the expected number of timing rows. The checksum is a program diagnostic rather than a cryptographic output signature. Because the local_run.csv file, full machine details and repeated execution results have not been supplied for inspection, no local latency estimate, reproducibility statistic or comparison to the pilot host is asserted here.

8. Discussion

8.1 What the results support

- BAP's descending power-of-two region decomposition has a compact mathematical representation and an XOR/MSB rule for locating the containing logical chunk.
- For the aligned starts and shared maximum logical chunk size in v2, BAP limited matches the strong idealized multi-size baseline rather than reducing its logical count.
- Virtual-alignment feasibility is not automatic: v5 shows that region-relative BAP chunks at unaligned starts cannot generally be inserted unchanged as native aligned leaves.
- Unbounded BAP provides a lower bound on *logical* chunk count, but is not an established implementation or TLB-reach upper bound on current hardware.
- Synthetic whole-chunk activation may induce a high loaded/touched ratio in sparse workloads; simplified splitting can reduce this ratio while adding entries.
- v6 defines and checks consistent byte-address translations across four software representation models while respecting page permissions and explicit PFN layouts; correctness is conditional on its reference region descriptors and idealized access path.
- Generic range baselines can equal or beat BAP in record count for contiguous regions, so XOR/MSB-based parent partitioning is not yet shown to improve total metadata space or lookup work.

- v7 demonstrates a negative metadata-space result for the declared packed layout: BAP is not smaller than either generic-range baseline in the tested cases. Fragmented layouts show fewer BAP child-list key comparisons, but this is not a complete lookup-cost or speed measurement.
- The v8 C++ pilot measures software-record lookup time and supplies a competitive indexed generic-range control. Its rankings differ by layout, and its modeled storage bytes remain no better for BAP in the examples reported; this is not a universal speedup or storage benefit.

8.2 What the results do not support

- The reported 21 region-relative BAP chunks at unaligned starts are not 21 legal native translations in either v5 page-size profile; conventional leaf support cannot realize the claimed difference without changes or fallback retiling.
- No experiment demonstrates that BAP reduces real TLB misses, page walks, translation latency, resident memory, or program runtime compared with existing hardware paging.
- The v3 split model does not include optimal legal buddy splitting, runtime split/merge costs, TLB shootdowns, allocator effects, or actual demand-paging/COW behavior.
- The feasibility audit checks necessary virtual eligibility only. It does not certify physical frame continuity, protection correctness, software integration, or novelty relative to other region/range translation designs.
- v6 does not establish hardware realizability, lower page-walk or TLB costs, compact total metadata, efficient invalidation, or a runtime speedup, despite passing software-model translation tests.
- The lower v7 BAP comparison count cannot be interpreted as a faster end-to-end translator because the measured operation excludes parent arithmetic and memory accesses, and region discovery is common but not instrumented.
- v8 local command completion confirms the instrumented semantic and 540-row test gate but does not by itself establish a stable or statistically significant latency advantage; the local CSV has not been analyzed.

8.3 Design implication

v6 established translation semantics in a controlled model, v7 found no metadata-space benefit under its packed layout, and the v8 native-code pilot revealed workload-dependent software lookup rankings once an indexed-range comparator was included. The most defensible next stage is a controlled non-virtualized replication that captures independent process runs, compiler/CPU configuration and confidence intervals, followed by update/invalidation costs and genuine access traces. Only after the software-level cost balance is established should a hardware/TLB integration proposal be evaluated. If BAP cannot exceed a comparably indexed range representation, the contribution should be narrowed to its mathematical chunk-selection rule and its specific operational tradeoffs.

9. Limitations and Threats to Validity

- No OS or hardware implementation: v6 constructs and validates virtual-to-physical translations inside a software reference model, but it does not modify kernel paging, install BAP leaves in commodity hardware, or measure a page walker or TLB.
- Optimistic TLB proxy: summing the largest logical mappings is not a TLB miss-rate simulation and ignores replacement, associativity and native leaf eligibility.
- Virtual vs physical constraints: v5 audits virtual eligibility; v6 additionally tests fixed, synthetic PFN continuity, alignment phases and permission patterns, but not physical allocator behavior, NUMA, compaction, migration or aliasing.
- Synthetic activation and splitting: whole-chunk loading is an assumed policy, not a necessary property of conventional page translation; estimated splitting omits implementation cost and legal page-tree structure.

- Synthetic traces and reproducibility: workloads record unique touched pages rather than full address-access sequences; v3 also mixes a randomized Python string hash into its RNG seed, which can affect cross-process reproducibility unless fixed.
- Hardware and novelty: the region-relative lookup cannot be interpreted as a native huge-page mapping without new support or legal retiling. Costs of metadata, region discovery and translation must be compared against prior region/range-based methods.
- Representation and cost comparability: parent chunks, range records and native page-table leaves are not the same architectural objects. v6 counts omit descriptor and index bytes, cache locality, region discovery cost, update frequency and actual ISA/hardware integration.
- v7 byte layout is hypothetical: fixed 32/24/8-byte records omit real pointer and allocator costs, global region indexes, address-width constraints and cache placement. Comparisons do not measure CPU cycles, page walks, or TLB operations; v7 does not include a kernel page-table byte baseline.
- v8 is a preliminary C++20 software metadata-search microbenchmark. Host virtualization and CPU frequency, cache state, allocation layout, compiler optimizations, and process scheduling may influence nanosecond-scale timings; neither CPU performance counters nor stable non-virtualized runs have yet been reported. The pilot excludes region discovery, dynamic map/protect/unmap/split/merge operations, TLB and page walks.
- The user-provided local execution screen shows successful semantic validation and 540 rows, but does not contain per-case measurements or full host configuration. Its results cannot be combined with the supplied virtualized pilot or used to assert cross-machine reproducibility until its output CSV is inspected.

10. Related Work

10.1 Superpages and huge pages

Superpages and huge pages are the closest practical relatives of BAP. They increase TLB reach by allowing one translation entry to map a larger memory region [1]. Linux HugeTLB provides explicit huge-page support and emphasizes the role of physically contiguous allocation [3]. Transparent Huge Pages attempt to automate huge-page use and include promotion/demotion behavior [2]. BAP differs by decomposing each region into binary chunks and using an XOR/MSB lookup rule instead of relying only on conventional fixed large-page sizes.

10.2 Multi-size page management

Multi-size page management attempts to use several page sizes dynamically. Trident, for example, studies transparent use of 1 GB, 2 MB, and 4 KB pages in Linux [5]. BAP limited is similar in spirit to multi-size management when the maximum chunk size is capped. The difference is that BAP defines the decomposition directly from the binary representation of the region size and uses a region-relative lookup rule.

10.3 Region-based address translation

Direct Segments reduce virtual memory overhead by mapping large contiguous virtual regions directly to contiguous physical memory using base, limit, and offset registers [4]. BAP is less aggressive: it remains page-compatible at the base-page unit and decomposes regions into multiple chunks rather than replacing page mapping with one segment.

10.4 TLB coalescing and contiguity-aware translation

Other work improves address translation by exploiting contiguity in memory mappings, including coalesced TLBs and subregion-contiguity approaches [6]. These approaches are related because they also try to increase the

coverage of translation entries. BAP approaches the problem from the virtual-region layout side by using binary decomposition and explicit chunk metadata.

10.5 Generic range representations as strong baselines

The v6 capped and unlimited generic range models encode contiguous, permission-uniform VA-to-PA runs without imposing BAP binary parent boundaries. They are deliberately abstract comparators: neither is presented as an implementation of an existing range-TLB architecture, Direct Segments [4], or the coalescing mechanisms discussed in [6]. The comparison indicates that BAP needs an explicit lookup/metadata-cost argument rather than an inference from parent-chunk count alone. Direct quantitative comparisons against published hardware range-translation methods remain future work.

The v7 results strengthen the case for generic contiguous ranges as a control rather than only comparing against fixed paging: their absence of forced binary-parent boundaries yields fewer record bytes in all tested layouts. The measured number of binary-search key comparisons is an implementation-dependent surrogate and must be compared against equally engineered indexing structures; v7 is not an empirical benchmark of Direct Segments, range TLBs or any published hardware scheme.

10.5.1 Relationship to indexed-range and range-translation work

v8 adds an indexed generic-range baseline to avoid attributing a speed benefit solely to BAP when bucketing can also reduce a global binary-search workload. This is a comparison with a modeled software representation, not an implementation or replication of published Range TLB or Redundant Memory Mappings (RMM) hardware. A rigorous originality assessment must compare full mechanism and costs against such prior range-translation schemes as well as against native paging.

10.6 Buddy allocation

Buddy allocation also uses power-of-two blocks and supports efficient split and merge. BAP is related to the buddy idea, but it applies power-of-two decomposition to virtual memory region mappings and address translation, not only physical memory allocation. The XOR/MSB lookup rule is the specific mechanism that gives BAP its distinct identity.

11. Conclusion

This study formulated BAP as a region-relative power-of-two decomposition with an XOR/MSB rule for locating logical chunks, followed by an explicitly modeled physical-frame and permission lookup. The correctness of these software-model mappings does not by itself imply compatibility with stock hardware page-table formats or improvements in translation performance.

Versions v2-v4 characterized logical chunk counts and synthetic chunk activation. v5 exposed native leaf-size and alignment prerequisites, v6 verified reference physical-frame translation and protection across 45 layouts, and v7 recorded 540 modeled-byte and search-key comparison rows. In v7, BAP used no fewer estimated metadata bytes than generic ranges. The v8 C++20 pilot moves from comparison counts to software lookup timing with an explicit indexed-range control. On the virtualized pilot host, BAP traded places with indexed ranges depending on physical fragmentation and permissions, while generally retaining a slightly larger modeled metadata footprint. An independent user run passed address/protection checks and produced 540 rows, but its actual measurement CSV was not examined.

Taken together, the experiments support a correct logical chunk-selection and software translation model with an exploratory, workload-dependent lookup tradeoff. They do not establish a robust speedup, TLB advantage, lower kernel-memory footprint or hardware deployability. Publication-grade evaluation requires a physically controlled

replication with uncertainty estimates, realistic access/update traces, fair cache/index costs and a clear comparison to existing range-translation designs.

References

[1] J. Navarro, S. Iyer, P. Druschel, and A. Cox. Practical, Transparent Operating System Support for Superpages. OSDI 2002. https://www.usenix.org/conference/osdi-02/practical-transparent-operating-system-support-superpages

[2] Linux Kernel Documentation. Transparent Hugepage Support. https://docs.kernel.org/admin-guide/mm/transhuge.html

[3] Linux Kernel Documentation. HugeTLB Pages. https://docs.kernel.org/admin-guide/mm/hugetlbpage.html

[4] A. Basu et al. Efficient Virtual Memory for Big Memory Servers. ISCA 2013. https://research.cs.wisc.edu/multifacet/papers/isca13_direct_segment.pdf

[5] V. S. S. Ram, A. Panwar, and A. Basu. Trident: Harnessing Architectural Resources for All Page Sizes in x86 Systems. MICRO 2021. https://www.csa.iisc.ac.in/~arkapravab/papers/MICRO21_Trident.pdf

[6] C. Yu, Y. Bai, and R. Wang. Enabling Large-Reach TLBs for High-Throughput Processors by Exploiting Memory Subregion Contiguity. arXiv:2110.08613, 2021. https://arxiv.org/abs/2110.08613

[7] A. Silberschatz, P. B. Galvin, and G. Gagne. Operating System Concepts. Wiley. Standard textbook reference for paging, segmentation, virtual memory, and memory-management background.

[8] Binary Adaptive Paging project repository, including simulators v2–v4, feasibility_v5.py and results_v5/feasibility_summary_v5.csv: https://github.com/abdojat/binary-adaptive-paging (repository state accessed 16 October 2026).

[9] BAP v6 experiment artifacts: translation_v6.py and results_v6/translation_semantics_v6.csv (software-model translation verification). See project repository [8].

[10] BAP v7 supplementary experiment artifacts: metadata_lookup_v7.py and results_v7/metadata_lookup_v7.csv (synthetic modeled metadata bytes and lookup-key comparisons; not measured memory usage or translation time).

[11] BAP v8 pilot artifacts: lookup_benchmark_v8.cpp, results_v8/lookup_latency_v8.csv, results_v8/replication_b.csv, results_v8/summary_s10000_start1_v8.csv and BAP_v8_Pilot_Research_Note.md (pilot native-code software metadata-lookup timing, not journal-ready data).

Appendix A. Reproducibility Notes

The archived v2-v4 experiments use their original scripts. v5 documents 72 virtual-eligibility cases; v6 verifies 45 translation cases; v7 reports 540 modeled-byte and key-comparison cases with 1,024 queries per row. v8 adds 540 C++20 software lookup timing cases per program execution under its default 4,096 queries and five repeated passes, across three seeds; two separate pilot executions are archived. The v3 Python process-randomized hash seed remains a reproducibility limitation. The user ran v8 locally and obtained the PASS message and 540-row summary, but the per-case CSV is not available in this manuscript revision. Preserve original compiler flags, full hardware configuration, exact result CSVs and code revision before publication.

```
python                  simulator_v2.py                  --validate                  --tlb-entries                 64
python               simulator_v3_workloads.py               --validate               --tlb-entries                 64
python                 sensitivity_v4.py                  --simulator-path                 simulator_v3_workloads.py
python          feasibility_v5.py           --validate           --out           results_v5/feasibility_summary_v5.csv
python          translation_v6.py           --validate           --out           results_v6/translation_semantics_v6.csv
python          metadata_lookup_v7.py           --validate           --out           results_v7/metadata_lookup_v7.csv
```

```
g++ -O3 -std=c++20 -Wall -Wextra -pedantic lookup_benchmark_v8.cpp -o lookup_benchmark_v8
./lookup_benchmark_v8 --out results_v8/local_run.csv --queries 4096 --repeats 5
```

The most important generated files were:

- fair_comparison_start0_v2.csv
- upper_bound_comparison_start0_v2.csv
- alignment_effect_v2.csv
- selected_workload_comparison_v3.csv
- sensitivity_aggregated_v4.csv
- fair_mapping_entries_v2.png
- upper_bound_mapping_entries_v2.png
- fault_amplification_sparse_random_v3.png
- fault_amplification_cow_burst_v3.png
- max_chunk_mapping_count_v4.png
- threshold_split_tradeoff_v4.png
- feasibility_summary_v5.csv
- translation_v6.py
- translation_semantics_v6.csv
- metadata_lookup_v7.py
- results_v7/metadata_lookup_v7.csv
- lookup_benchmark_v8.cpp
- results_v8/lookup_latency_v8.csv
- results_v8/replication_b.csv
- results_v8/summary_s10000_start1_v8.csv
- results_v8/local_run.csv (generated locally; not supplied for analysis)

The project repository has v7 code/results and the v7 Word/PDF manuscript at the revision reviewed. The v8 source and pilot outputs should be committed under their actual filenames and code revision before citing a public archive; the local_run.csv must be retained and analyzed separately from the virtualized pilot. The PASS line by itself is a correctness/execution check, not performance evidence.